

Question Analyzer

Right, most of this project just involved a lot of tweaking with the prompts rather than any real ml application and learning, but even api finetuning could be considered a pathway to learning how llms work

Detailed overview:

Let us look at this function by function:

From bottom to top we start at the base function:

`def create_complete_question_image:`

We create a 10x12 image grid and we divide the grid into 3 rows, the first row will contain the question, the second row will contain the image and the last row the answers. The analysis json text is wrapped and split into smaller lines and formatted properly to be added in the final image. The graphs(if available are added in the second row). Finally from the analysis we get the correct answers as a list if its multiple or just the single correct answer to add it to row 3

`def analyze_question_image:`

We get the image and feed an analysis prompt to the api that identifies the question text, whether a graph/diagram is missing or not, the answer choices and the problem with the question. The analysis is basically a json file that contain several important information about the question image like question_text, answer_provided, issue_type, current_state, what_should_be_shown, requires_graph_generation, requires_answer_correction, graph_specifications, correct_answers(either a list or a single answer)

`def generate_visual_correction:`

Now that we have the analysis and the incorrect graph or diagram, we generate visual corrections to it by feeding our api another prompt. We provide it the question, what should be shown and the graph specification from the analysis file and ask it to adhere to appropriate libraries depending on the diagram type.

`def process_question_image:`

This is one of the base functions, here initially we call the analyze_question_image function to get the analysis and from that analysis we see if it requires diagram correction, then we move on towards generate_visual_correction function and once the rectified diagram is given, we order it properly through the create_complete_question_image function that orders it and delivers a single image.

`def batch_process_questions:`

This basically runs the question processin pipeline on a whole bunch of images in a certain folder and also saves the summary and corrected diagrams of each question to a results folder.

EDIT:

Made changes to the prompts so that it will check thoroughly to see if the image is wrong or not.